



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Tema 5.1:

Aproximación de la

función de valor

Nota previa

- Vamos a ponernos con el RL de verdad
- Predicción y control con aproximación de las funciones de valor es un tema clásico en teoría de RL
- Por lo tanto, nos apoyamos en dos recursos “clásicos”: el libro de Sutton & Barto, y las diapositivas sobre aproximación de la función de valor de David Silver para UCL.

RL a gran escala

- Resolver problemas con espacios de estado muy grandes
- Ejemplos: Ajedrez, Go, navegación de vehículos autónomos, finanzas...



Aproximación de Funciones de Valor

- Problema: Los métodos tabulares son ineficientes (no escalan bien)
 - El problema no cabe en memoria
 - Hace falta demasiada experiencia para entrenar a un agente
- Problema: MDPs grandes con muchos estados y acciones.
- Solución: Aproximación de funciones $\hat{v}(s, w)$ o $\hat{q}(s, a, w)$

Tipos de Aproximación de Funciones

- Combinaciones lineales de características.
- Redes neuronales.
- Árboles de decisión, vecinos más cercanos.
- Bases de Fourier o Wavelet.
- Enfoque en aproximadores **diferenciables**.



Descenso del gradiente

- Método bien conocido por ser el estándar en entrenamiento de modelos de DL.
- Sea $J(w)$ una función vectorial derivable, cuyo gradiente es:

$$\nabla_w J(w) = \left[\frac{\partial J(w)}{\partial w_1}, \dots, \frac{\partial J(w)}{\partial w_n} \right]$$

- Podemos llegar a un mínimo local moviéndonos iterativamente en la dirección de máxima reducción de J desde un determinado punto:

$$\Delta w = -\frac{1}{2}\alpha \nabla_w J(w)$$

Aproximación con descenso del gradiente

- En nuestro problema J es:

$$J(w) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, w))^2 \right]$$

- Para la que la regla de actualización de los pesos va a ser:

$$\Delta w = -\frac{1}{2} \alpha \nabla_w J(w) = \alpha \mathbb{E}_{\pi} [(v_{\pi}(S) - \hat{v}(S, w)) \nabla_w \hat{v}(S, w)]$$

- Pero vamos a moverlos muestra a muestra

$$\Delta w = \alpha (v_{\pi}(S) - \hat{v}(S, w)) \nabla_w \hat{v}(S, w)$$

Representación del estado

- ¿Qué es el estado?
- ¿Qué tiene que cumplir una señal que nos transmita información sobre el estado?
- ¿Qué alternativas tenemos para “construir” esa señal?
- ¿Cómo vamos a llamarla? (Ejemplos)

Aproximación Lineal de Funciones de Valor

- Representación del estado con un vector de características $x(s)$ → Aproximación lineal:

$$\hat{v}(s, w) = x(s)^T w = \sum_j x_j(s) w_j$$

- Función de coste de DG cuadrática:

$$J(w) = \mathbb{E}_{\pi} \left[\left(v_{\pi}(s) - x(s)^T w \right)^2 \right]$$

- Regla de actualización de pesos:

Relación con caso tabular

- Caso especial de aproximación lineal.
- El vector de características es una codificación *one-hot* de los estados.
- w almacenaría los valores particulares de cada estado.
- ¿Consideras que hay algún tipo de aproximación?

Métodos Incrementales para la Predicción

- Hasta ahora, las fórmulas han contenido el valor real al que hay que aproximarse.
- Pero RL **NO** es aprendizaje supervisado. Sólo disponemos de la recompensa para guiar el aprendizaje.
- ¿Cuáles serán las fórmulas de actualización de pesos en la práctica para MC y TD?

Predicción MC con aproximación lineal

- El retorno G_t es un estimador no sesgado pero ruidoso del valor real del estado
- La interacción con el entorno nos va a dar muestras para entrenar usando DG estocástico:

$$\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, \dots, \langle S_T, G_T \rangle$$

- La predicción MC converge al óptimo global en el caso lineal, y es robusta frente al tipo de función con la que aproximemos el valor de estado.

Predicción TD con aproximación lineal

- El *TD-target* es un estimador sesgado (menos ruidoso que G_t) del valor real del estado
- La interacción con el entorno nos va a dar muestras para entrenar usando DG estocástico. ¿Qué pinta tienen?

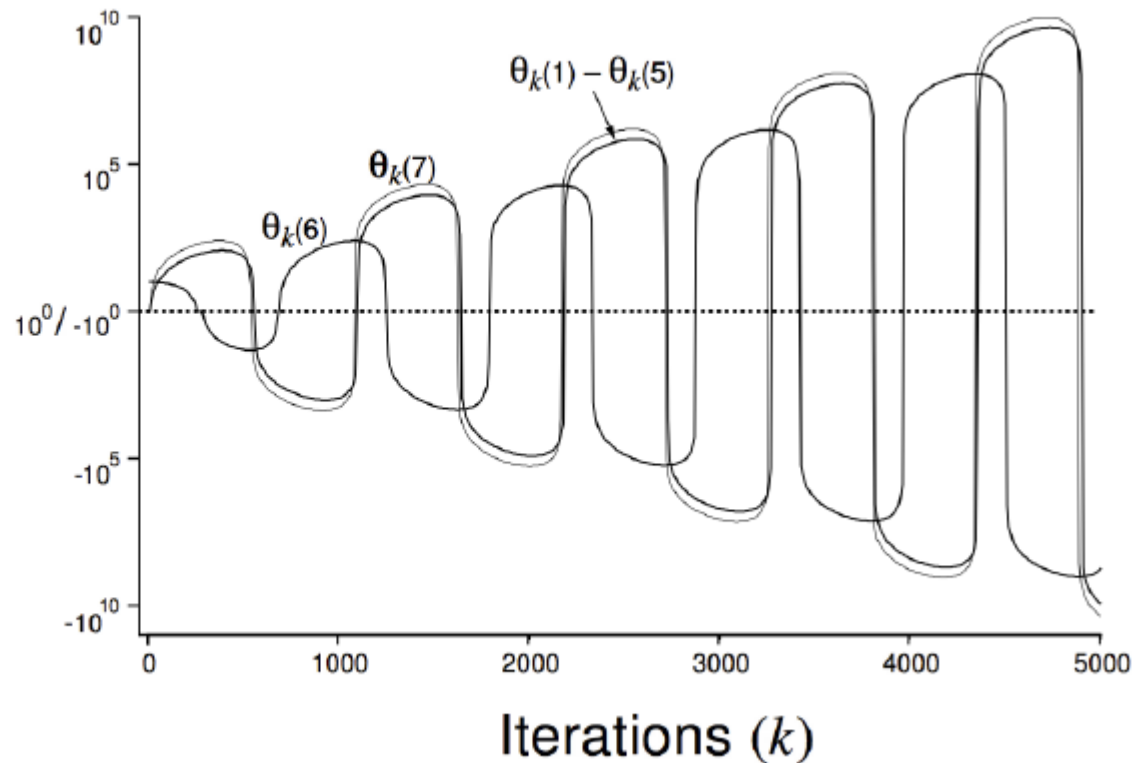
$$\langle S_1, R_2 + \gamma \hat{v}(S_2, \mathbf{w}) \rangle, \langle S_2, R_3 + \gamma \hat{v}(S_3, \mathbf{w}) \rangle, \dots, \langle S_{T-1}, R_T \rangle$$

- La predicción TD(0) converge a un punto cercano al óptimo global. Además, los algoritmos de predicción TD se pueden “romper” (sus pesos pueden divergir).

Consideraciones sobre convergencia

- Evolución de los pesos en el contraejemplo de Baird (predicción TD)

Parameter values, $\theta_k(i)$
(log scale,
broken at ± 1)

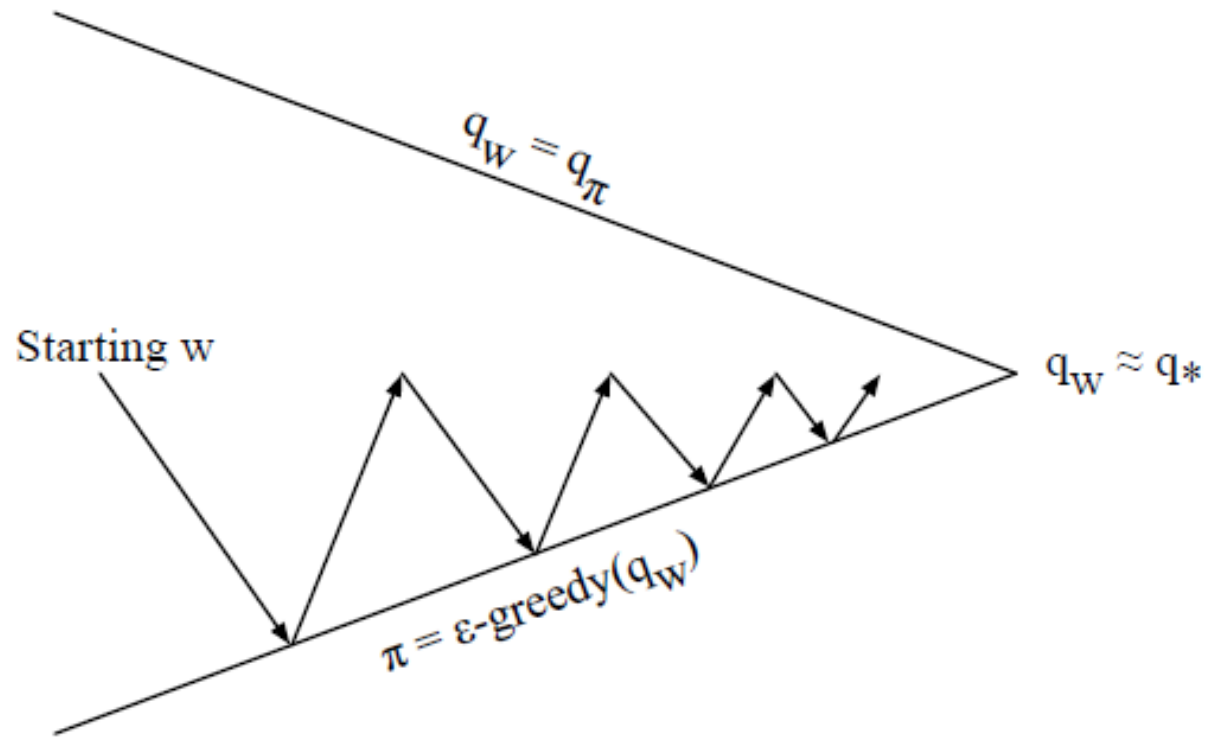


Consideraciones sobre convergencia (predicción)

Tipo de búsqueda	Algoritmo	Tabular	Lineal	No lineal
On-policy	MC	SÍ	SÍ	SÍ
	TD(0)	SÍ	SÍ	NO
Off-policy	MC	SÍ	SÍ	NO
	TD(0)	SÍ	NO	NO

- **Inductores de inestabilidad**
 - Aproximación de función de valor
 - *Bootstrapping*
 - *Off-policy*

CONTROL con aproximación



Paso de v a q

- ¿Cuáles serían las expresiones de la función aproximada, la de coste y la regla de actualización de pesos para aproximar q en lugar de v ?

$$\hat{q}(S, A, \mathbf{w}) \approx q_{\pi}(S, A)$$

$$J(\mathbf{w}) = \mathbb{E}_{\pi} [(q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w}))^2]$$

$$-\frac{1}{2} \nabla_{\mathbf{w}} J(\mathbf{w}) = (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

SARSA con Aproximación

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

If S' is terminal:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$$

Go to next episode

Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$$

$S \leftarrow S'$

$A \leftarrow A'$



Q-learning con aproximación

- Identifica las líneas que difieren de SARSA y define cómo serían para q-learning



Consideraciones sobre convergencia (control)

Algoritmo	Tabular	Lineal	No lineal
MC Control	SÍ	(SÍ)	NO
Sarsa	SÍ	(SÍ)	NO
Q-learning	SÍ	NO	NO

- **Inductores de inestabilidad**
 - Aproximación de función de valor
 - *Bootstrapping*
 - *Off-policy*

(SÍ) → cerca, pero no quieto

Trabajo para práctica 3

- Leer sección 9.5.4 (tile coding) del Sutton & Barto
- Realizar los ejercicios con el código que os vamos a entregar la semana que viene





COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Batch RL

comillas.edu

Motivación de batch RL

- El aprendizaje muestra a muestra tiene algunas ventajas
 - Sencillez
 - Bajos requerimientos computacionales
- Pero tiene también “defectos”
 - ¿Cómo es el **ciclo de vida** de la experiencia que nos llega?
 - Correlaciones entre muestras y entre objetivo y valor estimado → **convergencia--**
- Tenemos la opción de mover la función hacia el punto que mejor se ajuste a muchas muestras de experiencia → **batch RL**

La idea de predicción con mínimos cuadrados

- Volvamos por un momento al problema de predicción de $\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$
- Consideremos un conjunto de datos que recoge muestras de experiencia:

$$\mathcal{D} = \{\langle s_1, v_1^{\pi} \rangle, \langle s_2, v_2^{\pi} \rangle, \dots, \langle s_T, v_T^{\pi} \rangle\}$$

- Existe una familia de algoritmos que buscan el vector de pesos que minimiza el error cuadrático medio entre la aproximación y los valores de referencia:

$$\begin{aligned} LS(\mathbf{w}) &= \sum_{t=1}^T (v_t^{\pi} - \hat{v}(s_t, \mathbf{w}))^2 \\ &= \mathbb{E}_{\mathcal{D}} [(v^{\pi} - \hat{v}(s, \mathbf{w}))^2] \end{aligned}$$

Repetición de experiencia (experience replay)

- Usamos el *dataset* de experiencia:

$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$

- Pensemos en él como una distribución empírica de la experiencia obtenible, que podemos muestrear:

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

- Y aplicamos SGD a cada muestra para actualizar: $\Delta \mathbf{w} = \alpha(v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$
- Cuando repetimos el proceso suficientes veces, los pesos convergerán a:

$$\mathbf{w}^\pi = \underset{\mathbf{w}}{\operatorname{argmin}} LS(\mathbf{w})$$

Caso lineal

- En el caso particular de usar aproximación lineal, podemos calcular directamente el valor de w que minimiza el error cuadrático, evitando tener que iterar un número (que puede ser grande) de veces.
- La clave es pensar que en el mínimo de la función de pérdidas, los pesos no cambian:

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t) (v_t^{\pi} - \mathbf{x}(s_t)^{\top} \mathbf{w}) = 0$$

- En términos computacionales el tiempo para obtener w es $O(N^3)$, siendo N el número de variables. Pero hay una estrategia (Shermann-Morrison) para reducirlo a $O(N^2)$.

El caso de la DQN

- Resultados clave del equipo de Deep Mind en 2014 – 2015



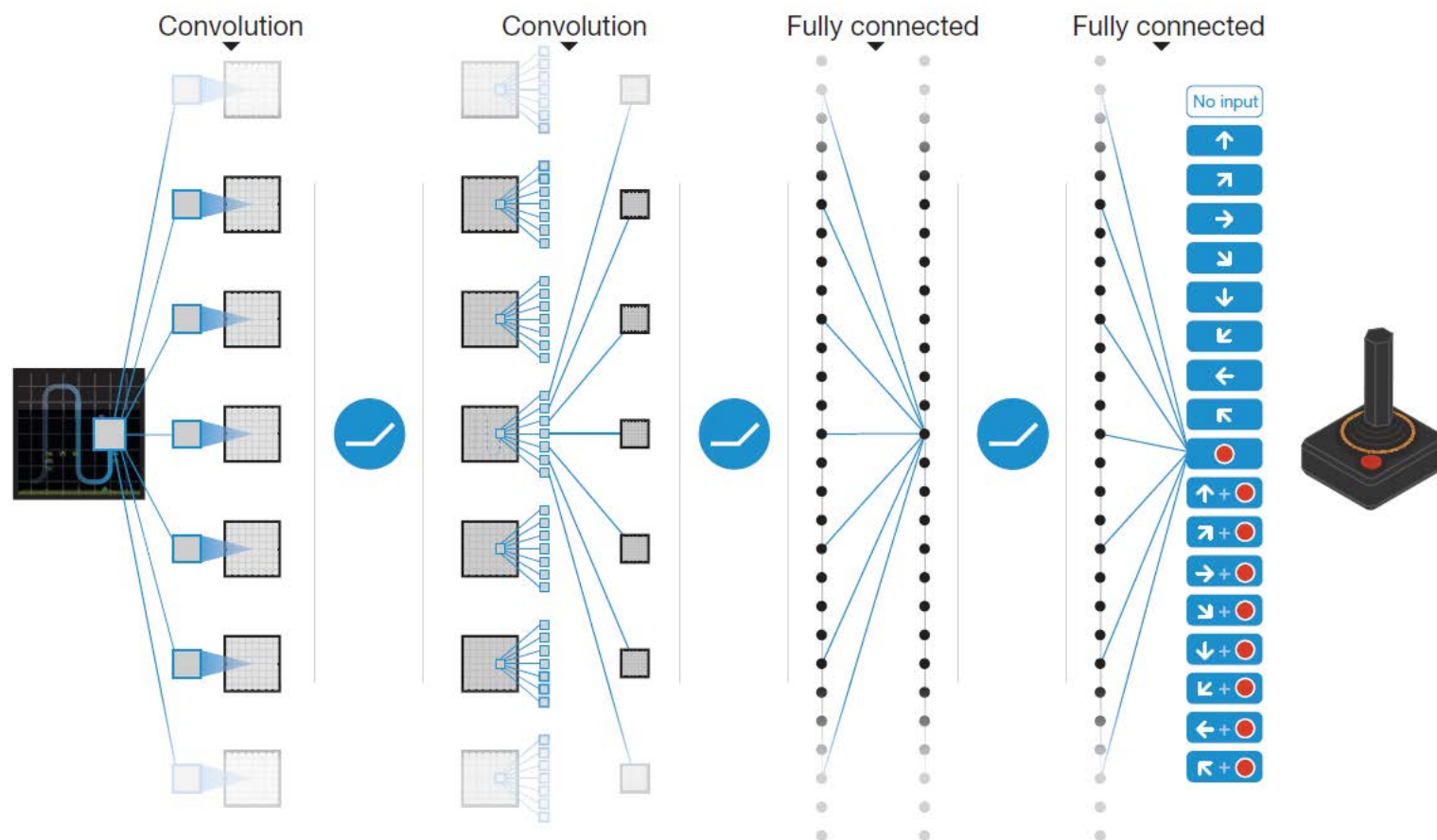
Objetivo y alcance de DQN

- Diseñado para dominar todos los juegos en la suite de Atari 2600 con una sola arquitectura de aproximador la misma estructura.
- Representación del estado (características) de cada juego está “en el tejado” de la red neuronal, i.e. aprende la representación y aproxima Q.
- Utiliza el mecanismo de repetición de la experiencia para:
 - Mitigar problemas relacionados con la rotura de la hipótesis iid y con la correlación entre objetivos y valores esperados.
 - Aumentar la eficiencia de la muestra

Modelo DQN

- **Observación del estado.** Recuerda que debe tener “suficientemente” la propiedad de Markov. Es la imagen actual devuelta por el emulador de videojuegos apilada con las 3 anteriores.
- **Acciones.** Todas las acciones válidas del joystick de Atari 2600.
- **Aproximador de Q.** Un modelo neuronal con 3 redes convolucionales + 2 redes FFFC + una salida lineal por acción (que aproximan los valores de acción para la entrada al modelo). Las activaciones son ReLUs.

Modelo DQN



Actualización de pesos

- **Función de pérdidas:** batch Q-learning con aproximación de función

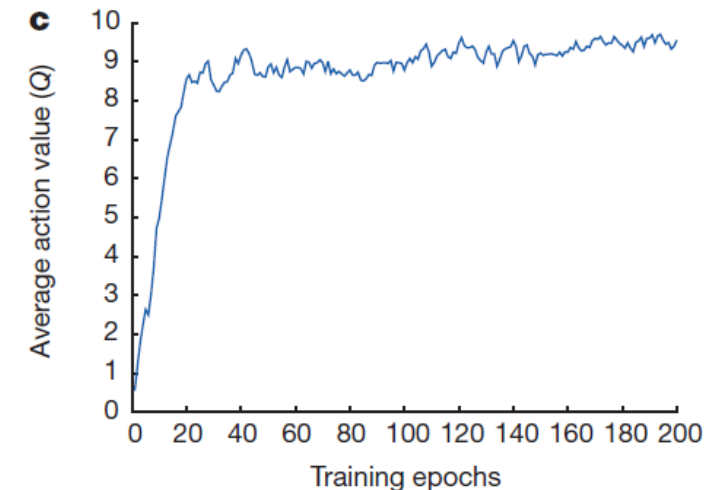
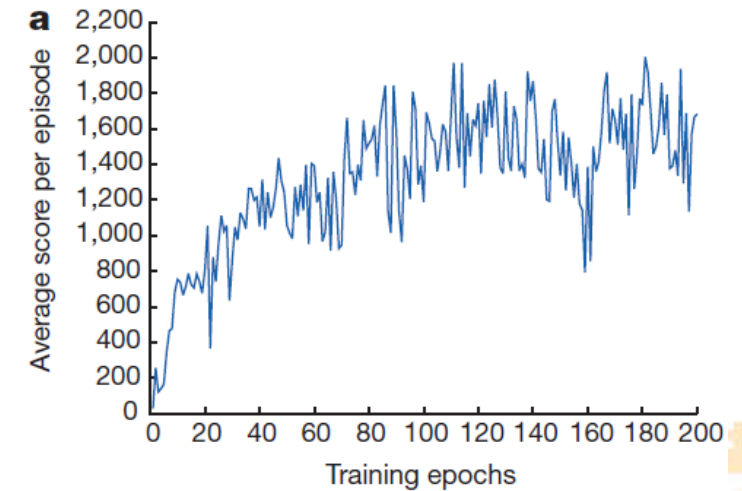
$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

- **Gradiente de la función de pérdidas:**

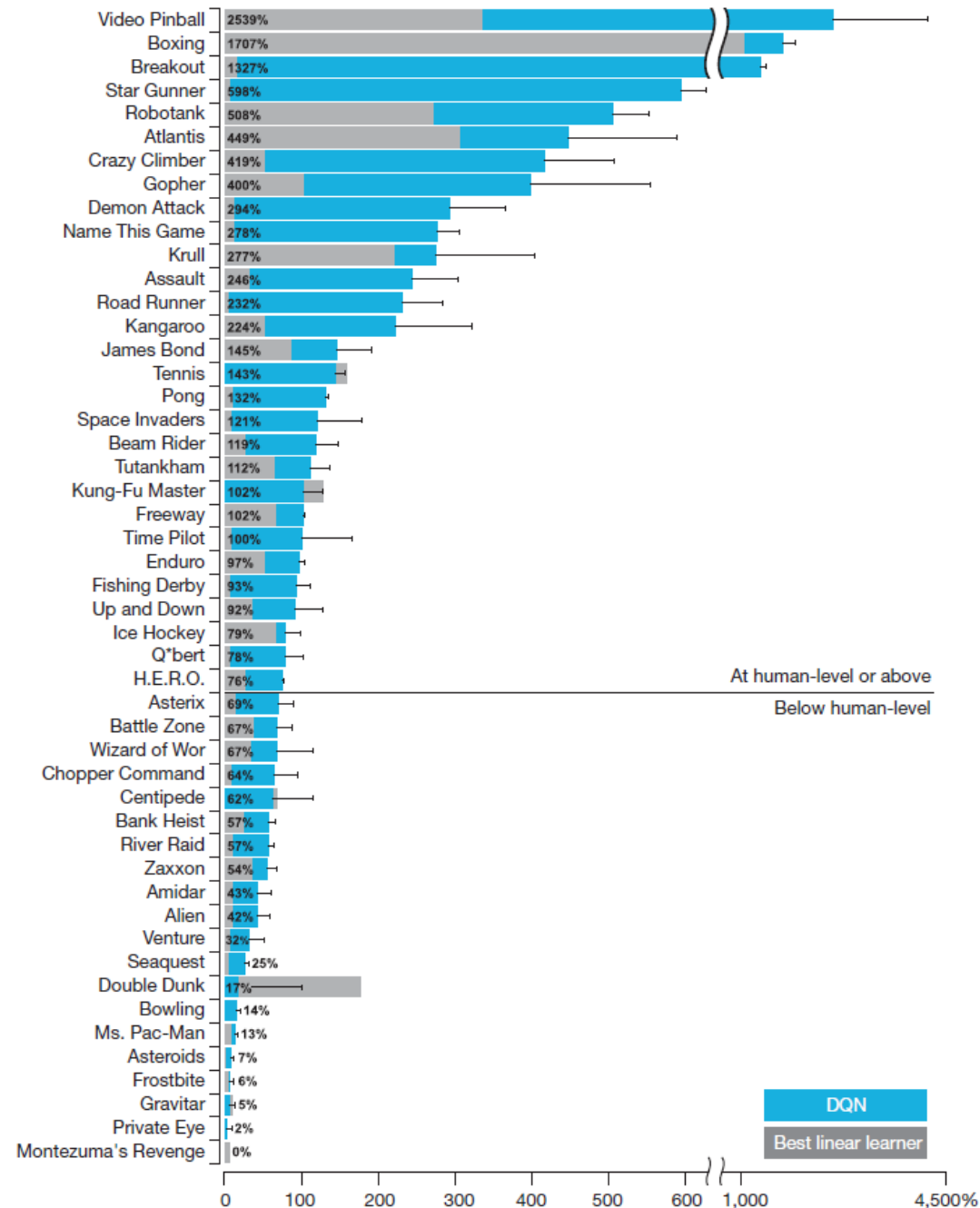
$$\nabla_{\theta_i} L(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

Resultados. Aprendizaje

- **Muy interesante:** definen indicadores de progreso de aprendizaje menos ruidosos que los más naturales (puntuación promedio).
- Esto es algo así como una “buena práctica” para monitorizar el proceso de aprendizaje.



Resultados. Rendimiento



Referencias

- Reinforcement Learning: An Introduction. R. Sutton, and A. Barto. The MIT Press, Second edition, (2018)
- Deep Mind – University College London Lecture Series: Introduction to Reinforcement Learning. (David Silver's slides)
- Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).
<https://doi.org/10.1038/nature14236>